

kakashi

Sovereign privacy infrastructure for agentic AI

Technical Implementation

Version 1.1.0 · September 2026

Author: Mohamed Atef Fahmy

Repository: github.com/Muhammadataf/kakashi

Package: npm @muhammadataf/kakashi

Licence: MIT

Developed in: United Arab Emirates

Prepared for: UAE AI Award — 3rd Edition

Category: Agentic AI Solutions Developed in the UAE

Contents

1.	Executive summary	10.	AI-agent integration protocol
2.	System architecture	11.	Trust boundary & threat model
3.	Core engine — detection & masking	12.	Test coverage & CI
4.	File-format handlers	13.	Deployment & operations
5.	Database masking layer	14.	UAE framework alignment
6.	PDPL compliance layer	15.	Roadmap
7.	Enterprise directory scanner	16.	Appendix — CLI reference
8.	Bilingual (EN / AR) i18n	17.	Appendix — pattern catalogue
9.	Agent-guard sidecar daemon	18.	Appendix — PDPL article map

1 · Executive summary

Kakashi is an open-source, locally-executed privacy layer designed to prevent personal data, credentials, and sovereign identifiers from crossing the boundary between a user's device and any external Large Language Model. It installs as a first-class *skill* inside more than 20 agentic-AI platforms — Cursor, Claude Code, GitHub Copilot, OpenAI Codex, Windsurf, Cline, Continue, Aider, JetBrains Junie, Roo Code, and others — and exposes a loopback HTTP API that any additional Model-Context-Protocol (MCP) client can consult before shipping data.

Kakashi is **not itself an artificial-intelligence system**. It is a deterministic, rule-based safety layer that operates around AI systems. This design decision eliminates every class of AI-ethics concern that stems from opacity, hallucination, learned bias, or model drift, and yields four inherent properties that regulated agentic-AI deployments require:

- **Deterministic** — the same input always produces the same masking output; no hallucinated tokens.
- **Auditable** — every detection rule is a regular expression readable by a compliance officer.
- **Bias-free** — no training data means no encoded bias; the same pattern fires for every user regardless of nationality, gender, or language.
- **Explainable** — every finding carries its pattern identifier, its severity, and the specific article of the UAE Personal Data Protection Law that governs it.

The single strategic sentence. Kakashi is the sovereign privacy infrastructure that makes agentic AI safe to deploy at UAE national scale. Every module described in this document exists to reinforce that thesis.

Version 1.1.0 highlights

Release date	September 2026
Detection patterns	35+ across three categories (ID & Documents, Personal Info, Credentials)
File formats	50+ with format-preserving masking

Database drivers	6 native adapters (PostgreSQL, MySQL, MongoDB, Snowflake, Databricks, SQLite)
Agentic platforms	20+ pre-integrated agent skills
PDPL articles mapped	9 (Federal Decree-Law 45/2021 Articles 1, 5, 6, 9, 15, 20, 21, 22, 25)
Languages	English + Arabic, full RTL preservation
Automated tests	101 across 8 test suites
Network calls	Zero during scan / mask; one to npm at install
Licence	MIT — open source

2 · System architecture

Kakashi is composed of four horizontal layers. Every layer is independently testable and consumes only the layer directly beneath it, giving the system the shape below.

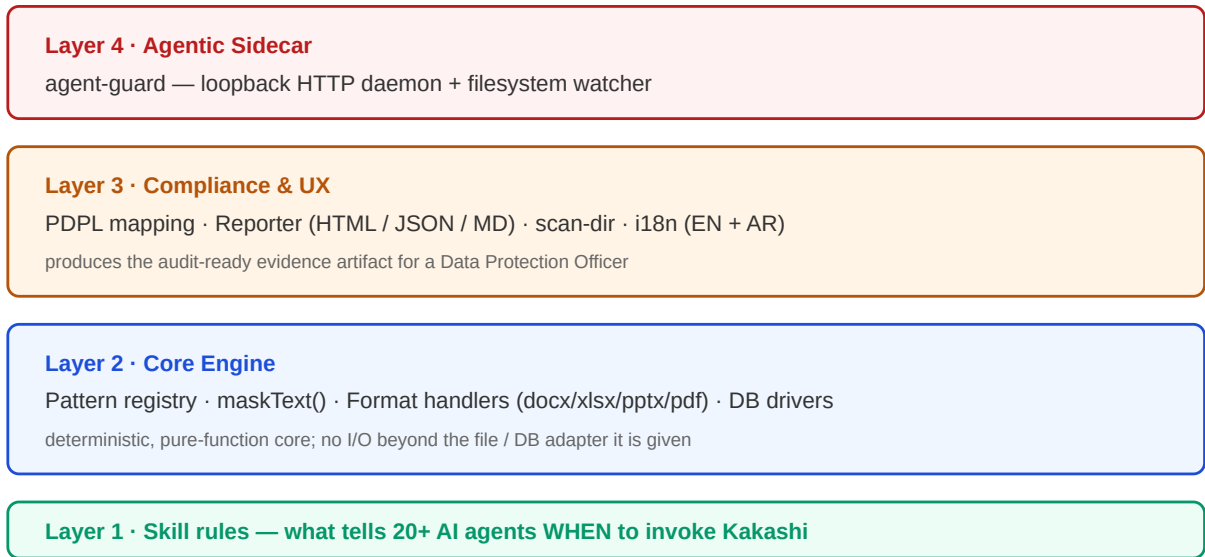


Fig 2.1 — Four-layer architecture. Each layer is a separate npm-package boundary and can be tested in isolation.

The layer boundaries are intentional. If an integration partner wants only the compliance reporter, they can import `src/lib/pdpl-mapping.js` and `src/lib/reporter.js` without adopting the CLI. If they want only database masking, `src/engine/db/` is a stand-alone module. If a government pilot wants only the agent-guard daemon, it ships as a single-command binary. Modularity is a first-class value.

Repository layout

```
kakashi/
├── bin/
│   ├── kakashi.js      CLI entry-point (Commander-based)
│   └── install.js      Agent-detection + skill installer
├── src/
│   ├── engine/
│   │   ├── patterns.js  Detection rules + checksum helpers
│   │   ├── masker.js    Tokenise + reconstruct
│   │   ├── formats/     Per-format read/write adapters
│   │   └── db/          Per-driver DB adapters (lazy-loaded)
│   ├── lib/
│   │   ├── pdpl-mapping.js UAE PDPL article map
│   │   ├── reporter.js   JSON / HTML / MD renderer
│   │   ├── scan-dir.js   Concurrent tree scanner
│   │   ├── i18n.js       EN + AR translations
│   │   ├── output.js     CLI printing
│   │   └── stats.js      Cumulative stats + impact snapshot
│   └── agent/
│       └── guard.js      Loopback HTTP daemon
├── tests/               101 automated tests, 8 suites
└── docs/               Architecture, submission, video kit
```

3 · Core engine — detection & masking

3.1 The pattern registry

Every detection lives in a single file, `src/engine/patterns.js`, as a plain JavaScript object with a stable schema:

```
{
  id:      'national_id',          // stable key -> token like [NATIONAL_ID_1]
  label:   'Emirates ID',          // English display name
  labelAr: 'الهوية الإماراتية',   // Arabic display name (for A4 i18n + HTML report)
  cat:     'id',                  // 'id' | 'pii' | 'cred'
  rx:      /\b784-\d{4}-\d{7}-\d\b/g, // regex
  validate: (m, text, i) => ...,   // optional false-positive filter
  fakeValues: ['784-1990-9999999-0'] // used by --mode fake
}
```

The `id` is stable across releases so a downstream user can rely on the token names (e.g. `[NATIONAL_ID_1]`) that appear in their masked files. The `label` and `labelAr` fields are free to evolve without breaking backwards compatibility. This separation is what allowed v1.1 to add Arabic labels to every existing pattern without breaking any existing masked artifact.

3.2 Checksum helpers

For the highest-stakes identifiers, Kakashi ships stand-alone checksum validators exported alongside the pattern registry:

- `luhnCheck(digits)` — the ISO/IEC 7812-1 Luhn algorithm.
- `isValidEmiratesId(id)` — 15-digit Luhn validation with mandatory `784` country prefix.

- `isValidIban(iban)` — ISO 13616 mod-97 remainder validation for International Bank Account Numbers.

These validators are deliberately *not* wired into the `validate` hooks. That decision reflects a threat-modelling principle: attackers frequently transpose digits, so a leaked identifier with a broken check digit is still a leaked identifier and must still be flagged. The stand-alone helpers are consumed by the reporter (§6) to *badge* findings as "checksum-verified" without discarding suspect-looking IDs.

3.3 The `maskText` algorithm

The masker is a single pure function with a five-step algorithm:

1. Collect all regex matches from every active pattern.
2. Sort matches by start position, then descending length (longer wins at the same position).
3. Resolve overlaps by keeping the first match whose start is beyond the last accepted match's end (non-overlapping set).
4. Assign consistent replacements — the same original value receives the same token throughout the document (`alice@x.com` always becomes `[EMAIL_1]`).
5. Apply replacements end-to-start so earlier offsets remain valid as later text is rewritten.

3.4 Three masking modes

MODE	EXAMPLE OUTPUT	WHEN TO USE
<code>typed</code> (default)	<code>[EMAIL_1]</code> , <code>[OPENAI_KEY_2]</code>	Preserves document readability and lets the AI reason about types
<code>redact</code>	<code>[REDACTED]</code>	Maximum anonymity when the AI does not need type context
<code>fake</code>	<code>user_a@example.com</code>	Preserves plausible LLM context for content-generation prompts

4 · File-format handlers

Kakashi supports over 50 file formats. The `src/engine/formats/` directory contains a handler per family, each implementing two functions:

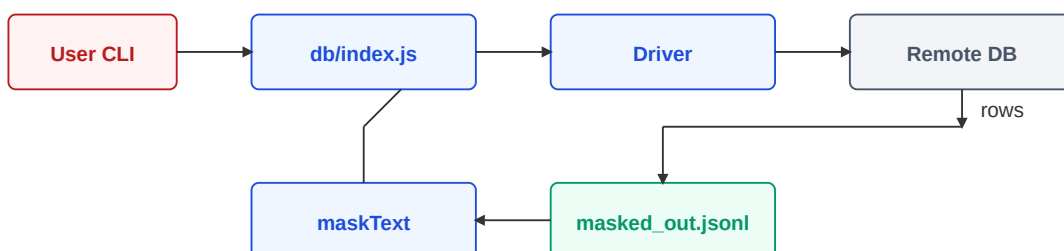
```
async function readFile(filePath) -> { format, text, ...metadata }
async function writeMasked(inPath, outPath, data, replMap, maskedText)
```

The `readFile` extracts a canonical text representation for the masker; the `writeMasked` reconstructs the file in its original binary format after substitution. This is what lets Kakashi output a real Word file, not a text dump of one.

FORMAT	LIBRARY	STATUS	NOTES
Text / code (40+ ext)	native <code>fs</code>	stable	Any text-shaped file; format is trivially preserved.
Excel (<code>.xlsx</code> , <code>.xls</code>)	<code>xlsx</code> (SheetJS)	stable	Cell-level masking; catches secrets embedded in narrative cells.
Word (<code>.docx</code>)	<code>jszip</code>	best-effort	XML <code><w:t></code> replacement; multi-run secrets tracked for v1.2.
PowerPoint (<code>.pptx</code>)	<code>jszip</code>	best-effort	XML <code><a:t></code> replacement.
PDF (<code>.pdf</code>)	<code>pdf-parse</code>	text-only	Text extract → masked <code>.md</code> ; real PDF round-trip tracked for v1.2.
JSON / JSONL / JSON5	native	stable	Text-based; masking preserves structure.
YAML / TOML / XML	native	stable	Text-based; masking preserves structure.
CSV / TSV	native	stable	Same code path as text; delimiters preserved.

5 · Database masking layer

Kakashi v1.1 introduces client-side database masking. A user can point Kakashi at a live database, run a SQL (or NoSQL) query, and receive a masked local copy of the results without ever exposing the raw rows to any AI agent. Six drivers ship out of the box.



All rows stream through the client-side driver; nothing traverses a Kakashi-owned server.

Fig 5.1 — Client-side database masking. Kakashi never brokers the connection through any hosted service.

5.1 Driver registry

Drivers are lazily `require()`-d, so a user only installs the client library for the database they actually consume. If the client is missing, Kakashi throws a friendly error naming the exact `npm install` command.

DRIVER	LIBRARY	INSTALL COMMAND
PostgreSQL	pg	npm install pg
MySQL / MariaDB	mysql2	npm install mysql2
MongoDB	mongodb	npm install mongodb
Snowflake	snowflake-sdk	npm install snowflake-sdk
Databricks	@databricks/sql	npm install @databricks/sql
SQLite	better-sqlite3	npm install better-sqlite3

5.2 CLI usage

```
# Count-only scan of a query – agent-safe, nothing written
kakashi db-scan "postgres://user:pass@host/db" -q "SELECT * FROM customers LIMIT 100"

# Mask a query result to a local JSONL file
kakashi db-mask "postgres://user:pass@host/db" \
  -q "SELECT * FROM customers WHERE country='UAE'" \
  -o masked_customers.jsonl -f jsonl
```

6 · PDPL compliance layer

The compliance layer, in `src/lib/pdpl-mapping.js`, links every detection pattern to specific articles of the *UAE Federal Decree-Law No. 45 of 2021 concerning the Protection of Personal Data (PDPL)*. The mapping is the intellectual centrepiece of Kakashi's regulatory value proposition.

Design choice. The file is *data, not logic*. It has zero I/O and zero side-effects. Consumers (the reporter, the agent-guard API, any third-party dashboard) import the map and enrich their findings in-process. Keeping this layer declarative means a legal counsel or compliance officer can review it as one flat document without reading application code.

6.1 Article catalogue

Nine articles are formally mapped:

ARTICLE	TITLE	KAKASHI COVERAGE
Art. 1	Definitions of Personal Data	All personal-info patterns
Art. 5	Conditions for Processing	Consent-preserving typed masking
Art. 6	Consent	Available for future explicit-consent workflows
Art. 9	Rights of the Data Subject	Available for right-to-erasure workflows
Art. 15	Sensitive Personal Data	Emirates ID, passport, IBAN, credit card, SSN
Art. 20	Security of Personal Data	Every credential pattern (15+ classes)
Art. 21	Reporting a Data Breach	Every leaked credential becomes a candidate breach event
Art. 22	Cross-Border Transfer	Every identifier destined for a foreign LLM
Art. 25	Data Protection Officer	The compliance report is DPO-ready

6.2 Severity heuristic

Every finding is scored:

- **CRITICAL** — sensitive personal data (Art. 15) OR live credentials
- **HIGH** — personal data with strong re-identification risk (emails, phones)
- **MEDIUM** — quasi-identifiers (names, dates of birth, IP)
- **LOW** — weak identifiers (age, ambiguous dates)

6.3 Public API

```
const { enrich, summarize } = require('kakashi/src/lib/pdpl-mapping');

// enrich a single finding
enrich(finding) -> { ...finding, severity, articles, articleSummaries, checksumVerified }

// aggregate a whole scan
summarize(findings) -> { findings, summary: { total, byCategory, bySeverity, byArticle, topArticle
```

7 · Enterprise directory scanner

The `scan-dir` subcommand walks a directory tree, honours `.gitignore` and `.kakashiignore`, scans every supported file concurrently, and emits an aggregate compliance report in JSON, HTML, or Markdown.

7.1 Concurrency model

Rather than adopting `worker_threads` and paying the overhead of spawning worker processes and marshalling large buffers over `MessageChannel`, Kakashi v1.1 uses an *async concurrency pool*. Because file scanning is I/O-bound, a Promise-based pool of size N yields most of the wall-clock benefit of true parallelism without the operational cost. Worker-thread parallelism for CPU-bound XLSX/PDF workloads is tracked for v1.2.

7.2 Report rendering

The reporter at `src/lib/reporter.js` is a pure DATA-IN, STRING-OUT module with three renderers:

RENDERER	CONSUMER	NOTES
<code>renderJson</code>	SIEM, CI, downstream tools	Machine-readable; every finding, every PDPL article, every timestamp.
<code>renderMarkdown</code>	CLI stdout, GitHub PR comments	Readable at a glance; suitable for a pipeline comment bot.
<code>renderHtml</code>	Audit-ready deliverable	Bilingual EN / AR; <code>dir="auto"</code> for RTL; <code>@media print</code> ready for <code>chrome --print-to-pdf</code> .

7.3 CLI usage

```
kakashi scan-dir ./project \  
  --format html \  
  --lang ar \  
  --parallel 8 \  
  --output audit_report.html  
  
# Then to convert to PDF (offline):  
chromium --headless --print-to-pdf=audit.pdf audit_report.html
```

8 • Bilingual (EN / AR) i18n

Kakashi ships a native Arabic experience — not a Google-Translate afterthought. The `src/lib/i18n.js` module carries English and Arabic string tables, resolves the active language from four sources in priority order, and provides a `t(key)` helper with placeholder substitution:

1. Explicit `--lang` CLI flag (highest priority)
2. `KAKASHI_LANG` environment variable
3. `LANG` environment variable if it starts with `ar` (e.g. `ar_AE.UTF-8`)
4. Fallback: English

8.1 What is translated

- All user-facing CLI strings — scan headers, finding summaries, error messages, success confirmations.
- Every pattern label — `label` and `labelAr` are stored side-by-side in the pattern registry.
- The full HTML compliance report — headers, table headings, PDPL article titles, severity labels, footer.
- PDPL article titles in Arabic — `title_ar` alongside `title_en` in `src/lib/pdpl-mapping.js`.
- A complete Arabic README at `README.ar.md`.

8.2 RTL preservation

Because Kakashi's replacement tokens (e.g. `[NATIONAL_ID_1]`) contain no bidirectional-embedding characters, the surrounding Arabic text preserves its natural right-to-left flow after masking. The HTML report declares `dir="auto"` so Arabic content renders RTL and English content LTR automatically within the same document.

9 · Agent-guard sidecar daemon

The `agent-guard` command in v1.1 is Kakashi's answer to the question that unlocks the "Agentic AI Solutions Developed in the UAE" category: *how do you make agentic AI safe to deploy at national scale?* The answer is to give every agent a local privacy sidecar it can consult before shipping any file to an external LLM.

9.1 Two roles

- 1. **Passive watcher.** On startup, agent-guard subscribes to filesystem change events under a user-specified directory. Every changed file is scanned; every finding is recorded in a JSONL audit log with UTC timestamps. A Data Protection Officer can review the log at any time and see what sensitive data was present in the tree.
- 2. **Active HTTP API.** agent-guard exposes three endpoints on `127.0.0.1` (loopback only — never a public interface). Any MCP-enabled agent, IDE plugin, or automation can call these synchronously before shipping data.

9.2 HTTP endpoints

VERB + PATH	BODY	RESPONSE
GET /health	—	{ ok, watching, uptimeMs, filesScanned, totalFindings, version }
POST /scan	{ "path": "..." }	{ path, summary } — PDPL-enriched summary; no raw values
POST /mask	{ "path": "...", "output": "..." }	{ output, replacements }

9.3 Loopback enforcement

Belt-and-braces security. Even though the daemon binds to `127.0.0.1` , every incoming request is additionally checked against `req.socket.remoteAddress` and rejected with 403 unless the origin matches a loopback prefix (`127.` , `::1` , `::ffff:127.`). This defends against edge cases where a reverse proxy or Docker network setup might have forwarded a non-local request to the daemon.

Zero-network-call preservation. agent-guard never opens an outbound socket. All state is in-memory plus one local JSONL log. Kakashi's "nothing leaves your machine" guarantee holds even while running as a long-lived daemon.

10 · AI-agent integration protocol

10.1 Shell-based agents

Six slash commands are installed by `bin/install.js` into the config directory of every detected agent on the host:

SLASH COMMAND	PURPOSE
<code>/kakashi</code>	Activate privacy mode for the session
<code>/kakashi-scan <path></code>	Counts only — agent-safe by default
<code>/kakashi-mask <path></code>	Write <code>masked_<file></code> alongside the original
<code>/kakashi-audit <path></code>	Full original → replacement mapping (deliberately verbose)
<code>/kakashi-stats</code>	Cumulative session statistics
<code>/kakashi-list</code>	Every active detection pattern

10.2 Supported agents (20+)

Claude Code · Cursor · GitHub Copilot · OpenAI Codex · Windsurf · Cline · Continue · Aider · JetBrains Junie · Roo Code · Kilo Code · OpenHands · Warp · Replit Agent · Augment Code · plus additional agents added on request.

10.3 MCP integration path

An MCP wrapper package (targeted for v1.2) will expose `kakashi_scan` and `kakashi_mask` as first-class MCP tools that any MCP-compliant agent (Claude Desktop, Cline, Continue, others) can call via the standard tool-calling protocol without any shell-out.

11 · Trust boundary & threat model

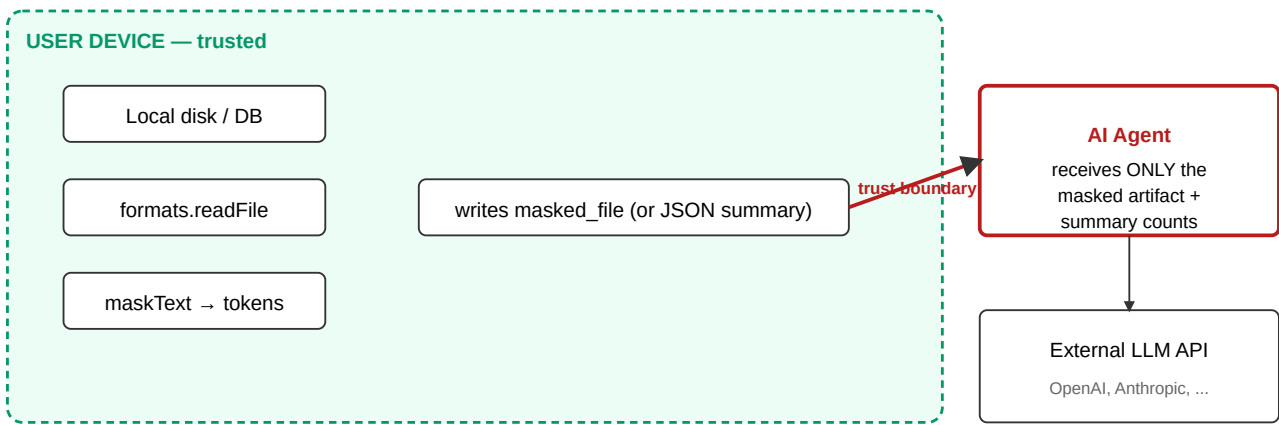


Fig 11.1 — The Kakashi trust boundary. Only masked artifacts and summary counts cross it.

11.1 Four defence-in-depth defaults

1. `scan` prints *counts only*; verbose per-finding previews require an explicit `--verbose`.
2. `mask` writes to disk; it never streams the masked bytes to stdout in the default flow.
3. `audit` is documented as deliberately verbose and is only invoked explicitly by the operator.
4. agent-guard's `/scan` API returns the PDPL summary — *never* the raw finding values, even over loopback.

11.2 STRIDE threat model

CATEGORY	THREAT	MITIGATION
Spoofing	Non-loopback caller impersonates a local agent	Loopback bind + <code>remoteAddress</code> hard-check
Tampering	Malicious patch disables a detection rule	Signed npm publisher, MIT-licensed source auditable line-by-line
Repudiation	User denies having exposed a credential	JSONL audit log + compliance report with UTC timestamps
Information Disclosure	Kakashi leaks the secrets it detects	Counts-only default; no telemetry; no error reporting
DoS	Enormous input file exhausts memory	Streaming per-row for DB, per-file try/catch for scan-dir, <code>--limit</code>
Elevation of Privilege	Daemon reads files outside its watched dir	Only reads paths the caller asks for; respects OS permissions

12 · Test coverage & CI

Kakashi ships with **101 automated tests** across 8 test suites. The suite is deterministic, hermetic, and runs offline in under three seconds.

SUITE	TESTS	COVERAGE
<code>patterns.test.js</code>	55	Every detection pattern, plus Luhn / Emirates-ID / IBAN checksum vectors
<code>masker.test.js</code>	6	All three masking modes, consistency, whitelist, line numbers
<code>cli.test.js</code>	4	Exit codes, mask-file round-trip, list-patterns
<code>pdpl.test.js</code>	6	Mapping drift-detection, article coverage, enrich, summarise, checksum badges
<code>db.test.js</code>	10	Driver inference, mock adapter, CLI db-scan / db-mask end-to-end
<code>reporter.test.js</code>	6	JSON, Markdown, HTML EN, HTML AR, <code>.kakashiignore</code> , no-raw-values invariant
<code>i18n.test.js</code>	8	Language resolution priority, <code>t()</code> substitution, live CLI EN + AR
<code>guard.test.js</code>	6	<code>/health</code> , <code>/scan</code> , <code>/mask</code> , 404, missing-path, raw-value-leak invariant

12.1 CI integration

```
- name: Kakashi privacy scan
  run: |
    npm install -g @muhammdatef/kakashi
    kakashi scan-dir . -f json -o kakashi-report.json
- name: Upload compliance report
  if: always()
  uses: actions/upload-artifact@v4
  with: { name: kakashi-report, path: kakashi-report.json }
```

Because `kakashi scan-dir` exits `1` when findings are present, the pipeline can be configured to *fail* pull requests that leak new sensitive data — turning Kakashi into a pre-merge safety gate.

13 · Deployment & operations

13.1 Install

```
# macOS · Linux · WSL · Git Bash
curl -fsSL https://raw.githubusercontent.com/Muhammdatef/kakashi/main/install.sh | bash

# Windows (PowerShell 5.1+)
irm https://raw.githubusercontent.com/Muhammdatef/kakashi/main/install.ps1 | iex

# Or via npm directly
npm install -g @muhammdatef/kakashi
```

13.2 System requirements

Runtime	Node.js ≥ 18
OS	macOS, Linux, Windows (native + WSL)
Disk footprint	< 15 MB core; drivers are lazy-loaded on demand
Memory (steady-state)	~40 MB agent-guard daemon; per-file scans are streaming
Network	Zero during operation; one call to npm at install

13.3 Uninstall

```
npx -y github:Muhammdatef/kakashi -- --uninstall # all agents
npx -y github:Muhammdatef/kakashi -- --uninstall --only cursor
```

13.4 Public dashboard

A public adoption dashboard is deployed via GitHub Pages, fetching live npm and GitHub metrics client-side without any Kakashi-owned backend. See `docs/dashboard/index.html`.

14 · UAE framework alignment

UAE FRAMEWORK	ALIGNMENT
UAE AI Strategy 2031	Enables the 50%-of-government-services agentic-AI programme by removing the compliance risk that otherwise blocks deployment. Kakashi is the trust layer beneath the strategy.
UAE AI Charter (10 June 2024)	Full alignment with all 12 principles. Kakashi is a rule-based safety layer, not itself an AI model, which inherently satisfies fairness, transparency, explainability, and accountability requirements.
UAE PDPL — Federal Decree-Law 45 of 2021	Every detection category explicitly mapped to specific articles (1, 5, 6, 9, 15, 20, 21, 22, 25) in <code>src/lib/pdpl-mapping.js</code> . The compliance report is audit-ready for the UAE Data Office.
UAE Data Office guidance	Default posture is "block the leak at source" — the strongest possible cross-border-transfer compliance stance.
UAE Centennial 2071	Sovereign, locally-developed, open-source (MIT) digital infrastructure. Directly serves the 2071 vision of homegrown sovereign technology.
UAE Cybersecurity Strategy	Zero-trust posture: no network calls, no cloud dependency, no telemetry, loopback-only daemon.
OECD Principles for Trustworthy AI	All five principles addressed: inclusive growth, human-centred values & fairness, transparency & explainability, robustness & safety, accountability.

15 · Roadmap

V1.2 (PLANNED Q4 2026)

- Native MCP server package (`@muhammdatef/mcp-kakashi`) — agent-guard endpoints as first-class MCP tools
- True `worker_threads` parallelism for 100 k+ file trees (CPU-bound XLSX/PDF workloads)
- Real PDF round-trip masking (content-stream patching via `pdf-lib`)
- Multi-run DOCX / PPTX secret handling (secrets that span format boundaries)
- Additional national identifier formats: KSA, Egypt, India, EU
- Streaming DOCX / PDF masking for gigabyte-scale documents

V1.3+ (RESEARCH DIRECTION)

- Optional local ML classifier for context-aware name detection (opt-in only, still 100% local)
- Proxy-mode masking that transparently intercepts existing app → DB traffic
- UAE Data Office direct submission integration for compliance report delivery

16 · Appendix — CLI reference

<code>kakashi scan <file></code>	Scan and report finding counts (agent-safe)
<code>kakashi mask <file></code>	Mask and reconstruct original format
<code>kakashi audit <file></code>	Full original → replacement mapping (verbose)
<code>kakashi mask-dir <dir> -r</code>	Mask all supported files recursively
<code>kakashi scan-dir <dir></code>	Enterprise directory scanner with PDPL report
<code>kakashi db-scan <conn> -q ...</code>	Scan DB query results (agent-safe)
<code>kakashi db-mask <conn> -q ...</code>	Mask DB query to jsonl / json / csv
<code>kakashi db-audit <conn> -q ...</code>	Verbose DB audit (deliberately verbose)
<code>kakashi agent-guard --watch <d></code>	Run loopback privacy daemon
<code>kakashi stats</code>	Cumulative session stats
<code>kakashi impact --write path</code>	Privacy-preserving impact snapshot
<code>kakashi list-patterns</code>	All active detection patterns

Global flags:

<code>--lang <code></code>	en ar
<code>--mode typed redact fake</code>	Replacement style (default: typed)
<code>--whitelist val1,val2</code>	Values to never mask
<code>--overwrite</code>	Replace original (asks confirmation)
<code>--stdin</code>	Read from stdin
<code>-v, --verbose</code>	Per-finding previews (NOT agent-safe)

Alias: k (e.g. k scan file.txt)

17 · Appendix — pattern catalogue

ID & DOCUMENTS (9)

`national_id` Emirates ID (with Luhn) · `intl_phone` UAE Phone · `passport` Passport · `visa_id` Visa Number · `trade_lic` Trade License · `pobox` P.O. Box · `non_latin_name` Arabic Name · `unified_id` Unified ID · `uae_iban` UAE IBAN (with mod-97)

PERSONAL INFO (9)

`email` Email · `phone` Phone · `ip` IP Address · `cc` Credit Card · `ssn` SSN / National ID · `dob` Date of Birth · `date` Date · `age` Age · `full_name` Full Name

CREDENTIALS (17)

`jwt` JWT Token · `ssh_key` SSH Private Key · `aws_key` AWS Key · `openai_key` OpenAI Key · `anthropic` Anthropic Key · `hf_token` HuggingFace Token · `gh_token` GitHub Token · `slack` Slack Token · `stripe` Stripe Key · `bearer` Bearer Token · `db_conn` Database Connection · `databricks_token` Databricks Token · `databricks_host` Databricks Host · `s3_uri` S3 URI · `env_secret` Env Secret · `hex_secret` Hex Secret · `-` extensible

18 · Appendix — PDPL article map (excerpt)

ARTICLE	TITLE (EN)	TITLE (AR)	SAMPLE SUMMARY
Art . 1	Definitions of Personal Data	تعريفات البيانات الشخصية	Any data relating to an identified or identifiable natural person.
Art . 5	Conditions for Processing	شروط معالجة البيانات الشخصية	Requires explicit consent or a defined lawful basis.
Art . 15	Sensitive Personal Data	البيانات الشخصية الحساسة	Special protections for biometric, health, identity documents.
Art . 20	Security of Personal Data	أمن البيانات الشخصية	Duty to implement technical and organisational measures.
Art . 21	Reporting a Data Breach	الإبلاغ عن انتهاك البيانات الشخصية	Duty to notify UAE Data Office within stipulated timeframes.
Art . 22	Cross-Border Transfer	نقل البيانات الشخصية عبر الحدود	Restricts transfer outside the UAE unless adequate protection exists.
Art . 25	Data Protection Officer	تعيين مسؤول حماية البيانات	Requires DPO for large-scale or sensitive processing.

Article summaries are condensed for engineering reference. The compliance officer remains responsible for consulting the original Arabic text of the Decree-Law, which is the legally binding version, and the executive regulations for final compliance sign-off.

Kakashi · MIT licence · Made in the United Arab Emirates

github.com/Muhammdatef/kakashi · npmjs.com/package/@muhammdatef/kakashi

Every quantitative claim in this document is independently verifiable against the public open-source repository.